

RAG: dar ao modelo *o material certo*

Um modelo não conhece os documentos da sua empresa, tem data de corte e pode inventar. A técnica **RAG** (*retrieval augmented generation*) resolve isso entregando a ele, a cada pergunta, os trechos relevantes de uma base de conhecimento. Esta aula mostra o pipeline, um RAG mínimo em Python (com o erro que ele comete e a correção), o que a Anthropic mediu com a recuperação contextual e quando **não** vale a pena usar RAG.

Pipeline

Embeddings

Busca

Limiar de relevância

Recuperação contextual

Avaliação

Formação Claude Code 2026 · Baseado em Lewis et al. (arXiv:2005.11401), no glossário e no artigo "Introducing Contextual Retrieval" da Anthropic e no OWASP LLM Top 10 2025, consultados em outubro de 2026 · Exemplo executado em [exemplos/](#)

O que o modelo *não sabe*

NÃO CONHECE A SUA EMPRESA

Política de reembolso, manual do produto, contratos: nada disso estava no treino (aula 04).

TEM DATA DE CORTE

O que mudou depois do treino, ele não sabe, e pode responder com informação desatualizada.

PREENCHE LACUNAS

Quando não sabe, tende a produzir algo plausível: a alucinação da aula 08.

A saída: em vez de esperar que o modelo "lembre", buscar a informação certa e colocá-la no prompt. É isso que o RAG faz.

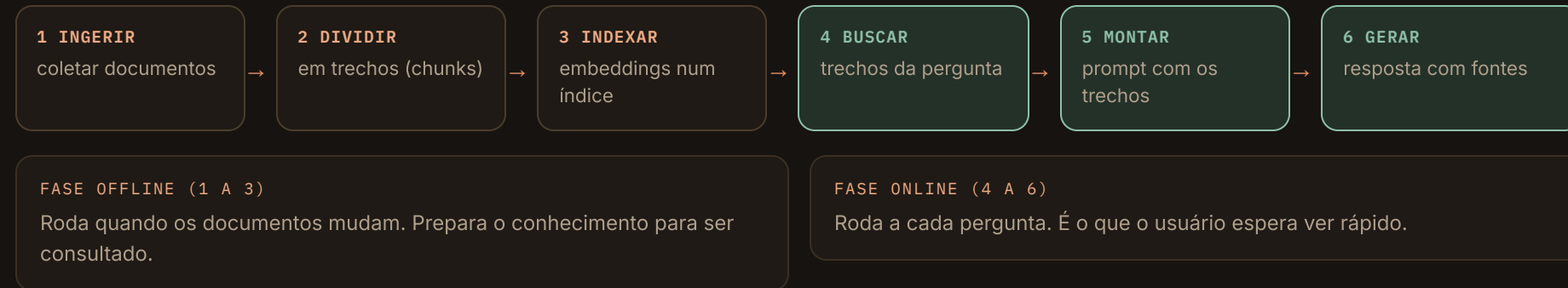
Memória nos pesos *e memória consultável*

- ✓ O artigo que deu nome à técnica (Lewis et al., 2020) combina dois tipos de memória: a **paramétrica**, guardada nos pesos do modelo, e a **não paramétrica**, um índice de documentos que ele consulta
- ✓ No trabalho original, o índice era um **índice vetorial denso da Wikipedia**, acessado por um recuperador neural
- ✓ Os autores relatam resultados de estado da arte em três tarefas de perguntas e respostas de domínio aberto

Definição de bolso (glossário da Anthropic): RAG combina recuperação de informação com geração de texto para melhorar a precisão e a relevância, e para **ancorar a resposta em evidência**; os documentos são passados na janela de contexto.

Fontes: arxiv.org/abs/2005.11401; docs.anthropic.com (glossário).

Seis etapas, *duas fases*



Buscar pelo *sentido*, não só pelas palavras

- ✓ Cada trecho vira um **embedding**: números que codificam o significado (aula 01)
- ✓ Os embeddings ficam num **banco de vetores**, que permite buscar por similaridade semântica
- ✓ Na consulta, a pergunta também vira embedding; os trechos mais próximos são adicionados ao prompt enviado ao modelo

PERGUNTA	BUSCA POR PALAVRAS	BUSCA POR SENTIDO
"Posso devolver o produto?"	Só acha trechos com "devolver"	Acha também "troca" e "reembolso"
"Quanto dura o conserto grátis?"	Não acha "garantia"	Acha "garantia de 12 meses"

Fonte do fluxo: anthropic.com/news/contextual-retrieval. Na prática, muitos sistemas combinam as duas buscas.

Um RAG mínimo, *sem bibliotecas*

rag_simples.py · transformar texto em vetor

```
PARADAS = {"a", "o", "os", "as", "e", "de", "da", "do", "dos", "em",
            "no",
            "na", "para", "com", "por", "que", "qual", "ate", "apos",
            "um"}

def tokens(texto, limpar):
    palavras = re.findall(r"[a-z0-9]+", texto.lower())
    if not limpar:
        return palavras
    # sem palavras vazias e com plural removido (reembolsos -> reembolso)
    return [p.rstrip("s") if len(p) > 3 else p
            for p in palavras if p not in PARADAS]

def vetor(texto, idf, limpar):
    c = Counter(tokens(texto, limpar))
    return {t: n * idf.get(t, 0) for t, n in c.items()}
```

A busca

```
def buscar(pergunta, limpar, minimo=0.2):
    idf, indice = montar_indice(limpar)
    q = vetor(pergunta, idf, limpar)
    pontos = sorted(((cosseno(q, v), n) for n, v in
                     indice.items()),
                    reverse=True)
    s, nome = pontos[0]
    return (nome, round(s, 2)) if s >= minimo else
    (None, round(s, 2))
```

- ✓ Cada trecho vira um vetor de pesos por palavra (TF-IDF); palavras raras valem mais
- ✓ A similaridade é o **cosseno** entre o vetor da pergunta e o de cada trecho
- ✓ Abaixo de um **mínimo**, a busca devolve "nada relevante"
- ✓ É didático: não usa embeddings de rede neural

A recuperação errada *estraga a resposta*

SAÍDA

```
PERGUNTA: Qual o prazo para pedir reembolso?  
busca ingenua: ('frete', 0.4)  
busca limpa:  ('reembolso', 0.25)  
PERGUNTA: Quanto tempo dura a garantia do celular?  
busca ingenua: ('garantia', 0.32)  
busca limpa:  ('garantia', 0.32)  
PERGUNTA: Qual a capital da Franca?  
busca ingenua: ('garantia', 0.32)  
busca limpa:  (None, 0.0)
```

- ✓ **Pergunta 1:** sobre reembolso. A busca ingênua trouxe "**frete**" (pontuou por casar palavras comuns como "o" e "para"); a busca limpa trouxe "**reembolso**"
- ✓ **Pergunta 2:** as duas acharam "garantia"
- ✓ **Pergunta 3:** fora do assunto. A ingênua devolveu "garantia" com 0,32 (resposta errada com cara de certa); a limpa devolveu **nada**

O que corrigiu: tirar palavras vazias ("para", "qual", "de") e o plural ("reembolsos" → "reembolso"). Se a busca erra, o modelo recebe o documento errado e responde com confiança sobre ele.

Instrução, *trechos e pergunta*

Modelo de prompt (do Guia da aula 16 do módulo 39)

System: Você é um assistente que responde APENAS com base nos documentos fornecidos. Se a informação não estiver nos documentos, diga 'Não encontrei essa informação na base.'

Contexto recuperado:

[Documento 1: Política de reembolso...]

[Documento 2: FAQ de atendimento...]

Pergunta do usuário: Qual o prazo para pedir reembolso?

- ✓ A **regra de recusa** ("se não estiver nos documentos, diga que não encontrou") é o que impede a resposta inventada
- ✓ Cada trecho vem **identificado** (nome, versão, página) para a resposta poder citar a fonte
- ✓ Separe os trechos com delimitadores: eles são conteúdo, não ordens (prompt injection, aula 08)
- ✓ Se a busca devolveu "nada relevante", nem chame o modelo: responda que não encontrou

Recuperação contextual: *o que a Anthropic mediu*

- ✓ Ao dividir um documento em trechos, cada trecho perde o contexto ("a receita da empresa cresceu 3%" — qual empresa? qual trimestre?)
- ✓ A **recuperação contextual** acrescenta a cada trecho uma pequena explicação do seu lugar no documento antes de indexar
- ✓ O método usa duas partes: **embeddings contextuais** e **BM25 contextual** (busca por palavras com o contexto acrescentado)
- ✓ A métrica foi a taxa de falha de recuperação nos 20 primeiros trechos (1 – recall@20), média de várias bases: código, ficção e artigos científicos

EMBEDDINGS
CONTEXTUAIS

–35%

5,7% → 3,7% de
falhas

+ BM25 CONTEXTUAL

–49%

5,7% → 2,9%

+ RERANKING

–67%

reordenando os
resultados

Fonte: anthropic.com/news/contextual-retrieval (setembro de 2024). Resultados dos testes da Anthropic, nas bases deles.

Às vezes, *basta colocar tudo no prompt*

- ✓ A própria Anthropic recomenda: se a sua base tem menos de **200 mil tokens** (cerca de 500 páginas), dá para incluir **tudo no prompt**, sem RAG
- ✓ Com *prompt caching*, reaproveitar um prompt longo entre chamadas reduz a latência em mais de 2 vezes e o custo em até 90% (segundo o artigo, setembro de 2024)
- ✓ Hoje as janelas são maiores (aula 01), o que amplia essa possibilidade

SITUAÇÃO	CAMINHO
Base pequena e estável	Colocar no prompt (com cache)
Base grande ou que muda sempre	RAG
Precisa de citação por trecho	RAG
Documentos com permissões diferentes	RAG com controle de acesso

Medir a busca *e a resposta*, separadamente

A BUSCA ACHOU O TRECHO CERTO?

Monte perguntas com o documento correto conhecido e conte quantas vezes ele aparece entre os primeiros resultados.

A RESPOSTA USOU O TRECHO?

Verifique se cada afirmação está sustentada pelos trechos e se o modelo recusa quando não há resposta.

SEGURANÇA DOS VETORES

O OWASP lista fragilidades de vetores e embeddings (LLM08): controle de acesso aos documentos, dados envenenados na base e vazamento entre usuários.

DOCUMENTOS COMO VETOR DE ATAQUE

Um documento pode trazer instruções escondidas (prompt injection, aula 08). Trate o conteúdo recuperado como não confiável.

Quem pergunta só pode receber trechos que **tem direito de ver**: filtre a busca pelas permissões do usuário, não depois da resposta.

Antes de colocar *em produção*

- ✓ Comece com um **conjunto de perguntas reais** e a resposta esperada (aula 03)
- ✓ Teste **tamanhos de trecho** e sobreposição: trechos pequenos perdem contexto; grandes diluem a busca
- ✓ Guarde **metadados** (documento, seção, data, versão) para filtrar e citar
- ✓ Defina como os documentos são **atualizados e removidos** do índice
- ✓ Use **limiar de relevância** e a regra de recusa
- ✓ Mostre as **fontes** ao usuário, para ele poder conferir
- ✓ Registre pergunta, trechos e resposta para investigar erros
- ✓ Reavalie quando trocar o modelo de embedding ou o modelo gerador

Teste o que *ficou*

1. Em RAG, o modelo recebe:

A) Todos os documentos da empresa

B) Os trechos mais relevantes para a pergunta

C) Só a pergunta

D) O resultado de um novo treino

B.

2. No exemplo, a busca ingênua trouxe "frete" para uma pergunta de reembolso porque:

A) Casou palavras comuns como "para"

B) O modelo errou

C) Faltou internet

D) A pergunta estava em inglês

A.

3. Para a pergunta "Qual a capital da França?" numa base de políticas, o certo é:

A) Devolver o documento mais parecido

B) Responder que não encontrou na base

C) Inventar

D) Mudar a base

B. Limiar de relevância e regra de recusa.

4. Uma base de 150 mil tokens, estável, pode:

A) Ir inteira no prompt, com cache

B) Nunca usar o modelo

C) Exigir fine-tuning

D) Ser ignorada

A. Abaixo de 200 mil tokens, a Anthropic sugere considerar essa opção.

A resposta é *tão boa quanto a busca*

RAG ancora o modelo nos documentos da empresa: dividir, indexar, buscar, montar o prompt e gerar com fontes. A qualidade depende da recuperação, que deve ser medida à parte, com limiar de relevância, regra de recusa e controle de acesso. Se a base é pequena, coloque tudo no prompt; se é grande ou muda sempre, use RAG.

Pipeline de 6 etapas

Embeddings

Limiar + recusa

-35% · -49% · -67%

< 200 mil tokens

Próxima aula → 10 · Agentes IA × Workflow

Fontes: arxiv.org/abs/2005.11401; docs.anthropic.com/en/docs/about-claude/glossary; anthropic.com/news/contextual-retrieval; genai.owasp.org/llm-top-10 (LLM08), consultados em outubro de 2026. O exemplo `rag_simples.py` usa TF-IDF e foi executado para esta aula.